Mix and Learn

Music, DJ mixing and the app that teaches them: from a single note to a rendered set, on desktop and Android

Ahmad Ammar

8 November 2026

Contents

1 Start here **3**

1.1 Run it on your computer . . . 3

1.2 The first visit . . . 3

1.3 The parts of the screen . . . 5

1.4 One library, remembered . . . 5

1.5 Both decks together . . . 5

2 Music from first principles **7**

2.1 Sound, frequency and pitch . . . 7

2.2 Notes, octaves and semitones . . . 7

2.3 Intervals and why some sound sweet . . . 8

2.4 Scales, keys and chords . . . 8

2.5 The Camelot wheel . . . 8

2.6 Timbre . . . 8

2.7 Beat, tempo, bar and phrase . . . 8

2.8 Indian terms: sargam, tala and sam . . . 8

3 How the app measures a track **10**

3.1 Seeing frequencies: the Fourier transform . . . 10

3.2 Tempo . . . 11

3.3 Key . . . 11

3.4 Loudness and brightness . . . 11

3.5 Structure . . . 11

3.6 Your four tracks . . . 11

4 Mixing: by ear, by science, by machine **13**

4.1 Beatmatching and its cost . . . 13

4.2 Keys after the speed change . . . 13

4.3 EQ, crossfade and loudness . . . 13

4.4 Vocals over music: mashups and stems . . . 13

4.5 How different scenes mix . . . 14

4.6 Automatic choices 14

5 Building and remaking a set **15**

5.1 Step by step in the app 15

5.2 Set 1: Alfaaz and Faded 16

5.3 Set 2: Socha Hai and It's the Time to Disco 16

5.4 Both sets together 17

5.5 A whole folder at once, and where to start 17

5.6 Hook to hook 18

5.7 Playlists: one per zip, combined when you want 18

5.8 A video for Instagram 19

5.9 The set file and remaking a mix 19

6 Inside the code **21**

6.1 How data flows 21

6.2 Tests 21

6.3 Adding a preset 22

6.4 Open-source parts and their science 22

7 The Android app, from nothing **23**

7.1 The ideas 23

7.2 What is already done in this repository 23

7.3 Your steps: from a fresh computer to the phone 23

7.4 Keeping desktop and phone in step 24

7.5 Designing for touch 24

7.6 Do I need a second guide? 25

1 Start here

Mix and Learn is one program that does three jobs. It *teaches*: lessons, a sound lab and an explanation for every control. It *measures*: load your own tracks and it finds their tempo, key, loudness, brightness and structure. And it *mixes*: it plans a set, explains every decision, renders the mix as an MP3 or WAV file, and saves a small set file from which the same mix can be made again.

It is a web app: plain HTML, CSS and JavaScript with no build step. On a computer it runs in any modern browser. On Android the very same files are wrapped into an app with Capacitor (Chapter 7). There is one code base, so the desktop and phone versions never drift apart.

1.1 Run it on your computer

You need Python (any recent version) to serve the files; nothing else.

```
git clone https://github.com/AhmadAmmar/djmixer.git
cd djmixer
python dev/serve.py
```

Your browser opens `http://localhost:8765`. The server watches every file in `web/` and the git branch: whenever you save a file, commit, pull or rebuild, every open page reloads itself, and the server tells the browser never to use a cached copy. You always see the latest version without pressing anything.

1.2 The first visit

The first time the app opens, a short tour points at each part in turn (Figure 1.1); the *Tour* button repeats it. After that, anything with a dotted underline or a dashed border has an explanation: hover over it with a mouse, or press and hold it on a touch screen. Each explanation has four parts: what it is, what it means musically, the science behind it, and the file and function in the code.

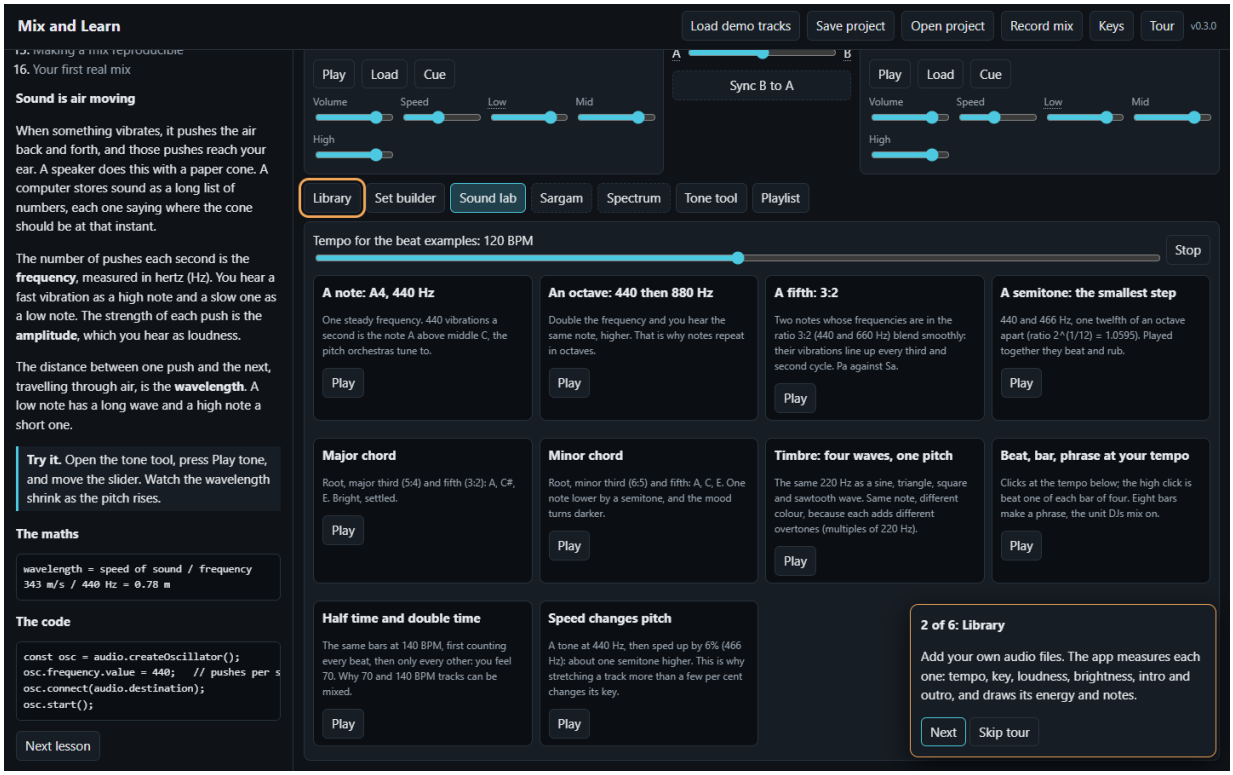


Figure 1.1: The first-run tour, here on its second step, the Library. The highlighted part has an orange outline; the box at the bottom right explains it.

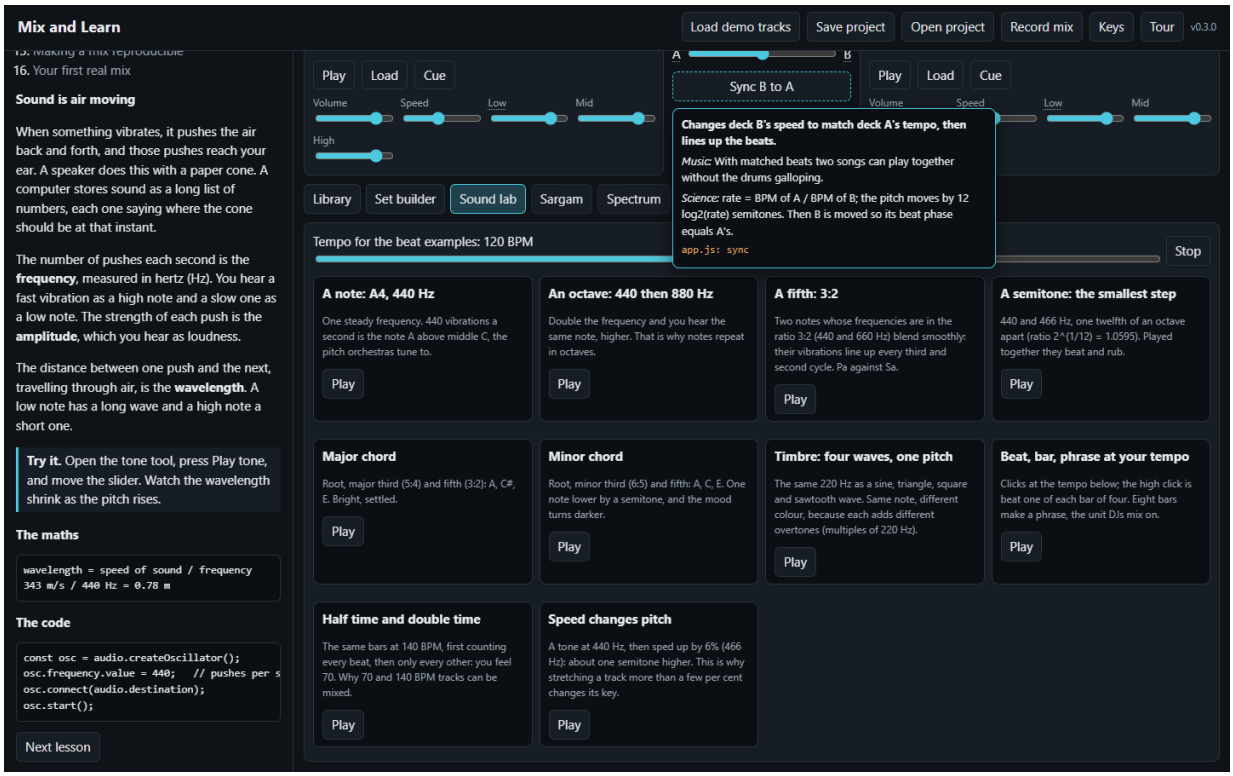


Figure 1.2: Hovering over *Sync B to A* opens its explanation: what it does, the musical reason, the formula, and where the code is.

1.3 The parts of the screen

Lessons (left) Eighteen short lessons, each with words, something to try, the formula and the code. They run from *sound is air moving* to *your first real mix*.

Decks A and B Two players, as on a DJ console. Each knob shows its value and has its own reset button (or double-click the slider):

- **Volume**, default $\times 1.00$ (0 dB): how loud this deck plays before the crossfader.
- **Speed**, default $\times 1.000$: shown also as a percentage and in semitones, because faster also means higher.
- **Bass** (below 250 Hz), **Mids** (around 1 kHz) and **Treble** (above 4 kHz), default 0 dB each, from -30 to $+6$ dB: the three equaliser bands. Bass is kick and bassline, mids are voices and melody, treble is hi-hats and the shimmer of strings.
- **Play**: the whole track, vocals only or music only (approximate, see Chapter 4).

Between the decks: the crossfader, *Sync B to A*, *Swap A* $\leftrightarrows$ *B* (exchanges tracks, positions and settings), $A \rightarrow B$ and $B \rightarrow A$ (put the same track on the other deck), and *Reset both decks*. Load a deck with its *Load* button, by dropping an audio file from a folder onto it, or by dragging a track's card from the Library.

1.4 One library, remembered

Every track, however it arrives (the Library button, a drop on a deck, or a drop on the Library), becomes one record in the Library, measured once. The decks, the Library and the Set builder all point at those same records: a track dropped on deck A appears in the Library marked *on deck A*, and *Use the deck tracks* in the Set builder turns the two deck tracks into a set. The records, with the original file bytes and the measurements, are kept in the browser's own database (IndexedDB) on this device, so they survive reloads and app updates, and the decks reload the tracks they last held. *Forget* removes a track. The memory belongs to the address: clearing the browser's data for it, or opening the app at another address or port, starts empty.

1.5 Both decks together

Below the decks, a strip shows both decks' beats on one time line, with now as the centre line: tall marks are beat one of each bar. When A's and B's marks sit on top of each other the beats are in step; the text gives the gap in milliseconds, which deck is ahead, how fast they drift apart, and each deck's volume. *Play both* starts the decks on the same tick of the audio clock.

Advise the mix (it also runs whenever a deck changes) explains how to mix A into B: where to leave A and why, what speed, start point and EQ B needs and why, and the style for the two together, with its rule. *Apply to decks* sets those values; *Play the transition* performs it, with every move timed on the audio clock. Under it, the best next tracks from the Library are listed with their reasons, ready to load on deck A, because DJs alternate decks. *Hide lessons* (top left) gives the decks the whole width.

Library Add audio files; each is measured and drawn. A table shows how well every pair would mix.

Set builder Choose tracks, a transition style and an order, then render.

Sound lab Hear the ideas: a note, an octave, a fifth, chords, timbre, beats and bars, half time.

Sargam, Spectrum, Tone tool Live views of pitch against the swaras, of all frequencies at once, and of a single tone.

The *Theme* button switches between light, dark and auto, which follows your computer or phone setting; the choice is remembered. Every colour, including those drawn on the waveforms and graphs, comes from the theme, so nothing disappears against the background.

Nothing you load leaves your device: files are read by the browser itself, and no audio is copied into project or set files. That matters both for privacy and because commercial music may not be shared.

2 Music from first principles

Every term a DJ uses has a physical meaning and a number behind it. This chapter builds them up in order; the Sound lab (Figure 2.1) plays each one.

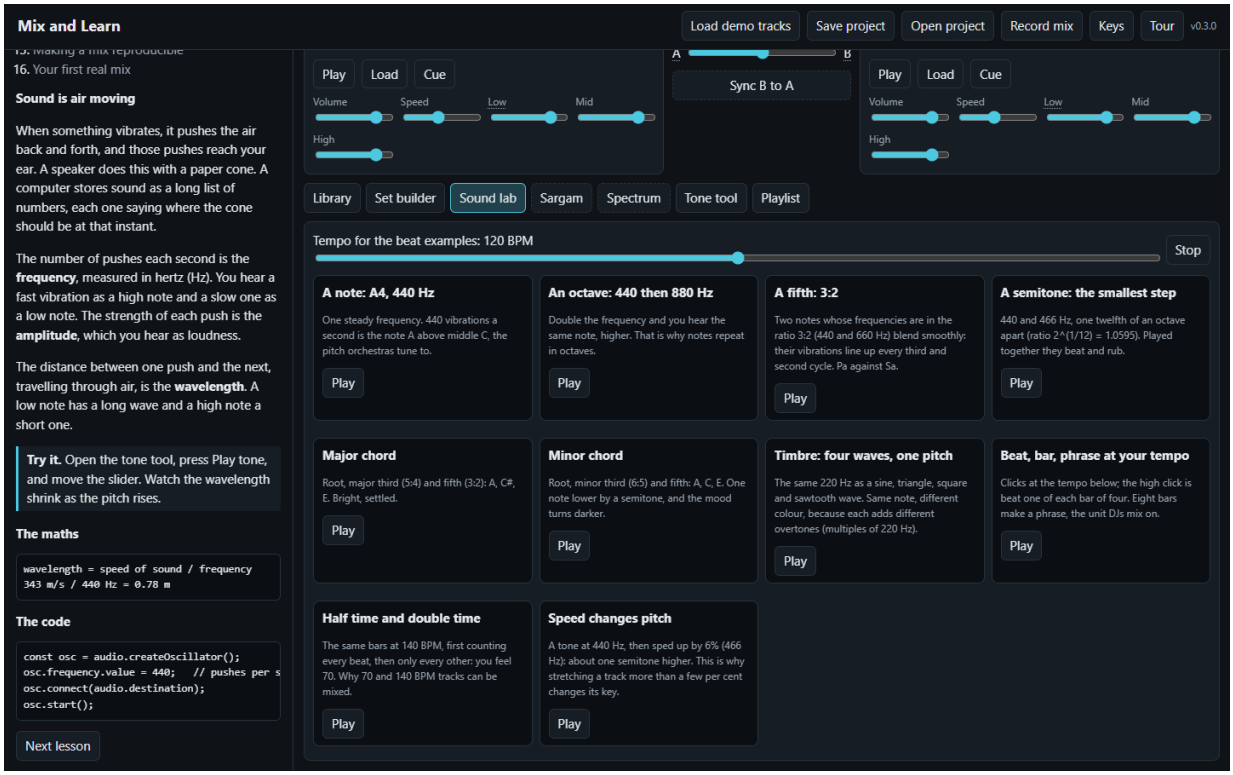


Figure 2.1: The Sound lab. Each card plays one idea and says in a sentence what to listen for.

2.1 Sound, frequency and pitch

Sound is air pushed back and forth. The number of pushes per second is the **frequency**, in hertz (Hz); we hear it as **pitch**, high or low. The size of the pushes is the **amplitude**, heard as loudness. Sound travels at about 343 m/s, so a wave's length is $\lambda = 343/f$: the A at 440 Hz is 0.78 m long, a bass note at 55 Hz over 6 m.

2.2 Notes, octaves and semitones

Doubling a frequency gives the same note an **octave** higher: the ear hears 220, 440 and 880 Hz as A. Western music divides the octave into 12 equal steps, **semitones**, each multiplying the frequency by $2^{1/12} \approx 1.0595$. Notes are numbered (MIDI numbers, A4 = 69), so

$$f = 440 \times 2^{(n-69)/12}\ \text{Hz}, \qquad n = 69 + 12\log_2 \frac{f}{440}.$$

A **cent** is a hundredth of a semitone, small enough to describe a singer slightly flat.

2.3 Intervals and why some sound sweet

Two notes whose frequencies form a simple ratio line up often: in a **fifth**, 3:2, every third vibration of the upper note meets every second of the lower. The ear hears such combinations as smooth (consonant). A semitone, ratio 1.0595, almost never lines up and rubs (dissonant). A **major third** is close to 5:4, a **minor third** to 6:5. In sargam, Sa to Pa is the fifth.

2.4 Scales, keys and chords

A **scale** is the set of notes a piece mainly uses. A major scale takes steps of 2,2,1,2,2,2,1 semitones; the natural minor 2,1,2,2,1,2,2. The **key** names the home note and the scale, such as A minor. A **chord** is notes sounded together: root, third and fifth make a triad, major (bright) or minor (darker) depending on the third.

Every major key has a **relative minor** using exactly the same seven notes (C major and A minor). This is why computers, and people, sometimes confuse them: the notes are identical and only the sense of home differs.

2.5 The Camelot wheel

DJs use the **Camelot wheel**, which numbers the 24 keys like a clock: 1 to 12, with A for minor and B for major. Neighbours differ by one note, so they mix smoothly; the same number with the other letter is the relative major or minor; a diagonal move (one number and a change of letter) shares most notes and changes the mood. The app scores these moves: same key 1, relative 0.9, neighbour 0.8, diagonal 0.6, two steps 0.5, further apart less.

2.6 Timbre

Real instruments vibrate at a main frequency and at multiples of it, the **overtones**. Their mix is the **timbre**, why a flute and a sitar playing the same Sa sound different. In the Sound lab the same 220 Hz is played as a sine (no overtones), triangle, square and sawtooth (strong overtones).

2.7 Beat, tempo, bar and phrase

The **beat** is the pulse; the **tempo** is beats per minute (BPM). Beats group into **bars** (mostly 4), bars into **phrases** (8 or 16 bars). At tempo B one bar lasts $4 \times 60 / B$ seconds: 2 s at 120 BPM. Songs change section on phrase boundaries, so transitions should too. Tempos twice apart (70 and 140 BPM) share every other beat: **half** and **double time**.

2.8 Indian terms: sargam, tala and sam

Sargam names notes by their distance from a chosen *Sa*: Sa Re Ga Ma Pa Dha Ni, with komal (flat) Re, Ga, Dha, Ni and tivra (sharp) Ma, twelve steps in all. A **tala** is a rhythmic cycle, such as teentaal of 16 beats, and its first beat, the **sam**, is where the cycle lands and the ear expects arrival. Bollywood

songs often combine a pop four-beat bar with a dhol or tabla pattern; the sam is the natural place to change songs.

3 How the app measures a track

A track arrives as a list of numbers: 44,100 samples per second per channel, each the position of the speaker cone. Everything the Library shows (Figure 3.1) is computed from those numbers by the functions in `web/dsp.js`, with no outside library. The app first averages the channels to one and keeps every fourth sample, about 11,000 per second, which keeps all frequencies up to about 5.5 kHz and makes analysis four times faster.

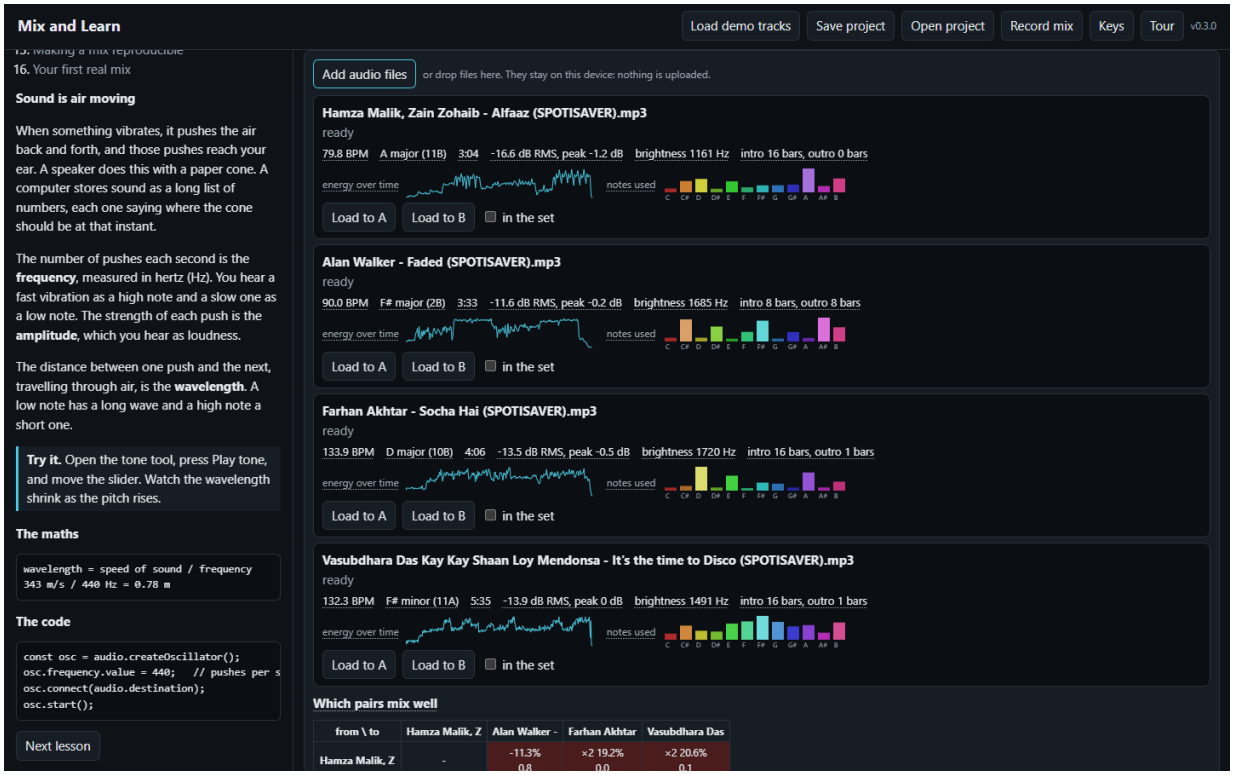


Figure 3.1: The Library with four tracks. Each card shows tempo, key, length, loudness, brightness, intro and outro, the energy over time and the notes used.

3.1 Seeing frequencies: the Fourier transform

A short slice of sound is a mixture of frequencies. The **Fourier transform** says how much of each it contains:

$$X_k = \sum_{n=0}^{N-1} x_n \, e^{-2\pi i k n / N}, \qquad f_k = k \, \frac{f_s}{N}.$$

The app uses the fast algorithm (FFT) on slices of 1,024 to 8,192 samples, each first multiplied by a smooth *Hann window* so the slice's edges do not create false frequencies.

3.2 Tempo

1. **Onsets.** For each 23 ms frame, take the spectrum and add up every *increase* since the previous frame. Drums and new notes make sharp rises; this is the onset strength.
2. **Repetition.** Music repeats, so the onset curve matches itself one beat later. The **autocorrelation** $R(L) = \sum_i e_i\, e_{i+L}$ is large when L is the beat gap. A gentle preference for gaps near 120 BPM chooses between a tempo and its half or double.
3. **Between frames.** At 43 frames a second, one frame near 130 BPM is 7 BPM, too coarse. So the curve is read between frames by straight-line interpolation, and compared with itself four beats later, where a small error has grown fourfold. On synthetic tracks of known tempo the result is within 0.4 BPM (tested in `web/test_dsp.mjs`).
4. **Phase.** The first beat is the offset at which onsets, summed every beat, are strongest.

3.3 Key

The **chroma** vector adds the energy of every frequency to its note name (C, C#, …, B), whatever the octave. It is then compared, by correlation, with the Krumhansl–Kessler profiles (how strongly each note belongs to a key, measured in listening experiments) rotated to each of the 24 keys. The best match is the key; the correlation is its confidence.

3.4 Loudness and brightness

RMS level is $\sqrt{\frac{1}{N}\sum x_n^2}$, in decibels $20\log_{10}$ of it, relative to full scale (dBFS). The **peak** is the largest sample. A peak above 0 dBFS means the decoded file overshoots full scale: heavy mastering plus MP3 decoding. **Brightness** is the spectral centroid $\sum f_k|X_k| / \sum |X_k|$.

3.5 Structure

Loudness is computed for every second and averaged over 8-bar phrases from the first beat. Phrases above half the loudest are *full*; the intro is the bars before the first full phrase, the outro the bars after the last. These are where DJs mix.

3.6 Your four tracks

Measured by `tools/plan.mjs`, which decodes with ffmpeg and runs the same `dsp.js`:

Track	BPM	Key	RMS (dB)	Length
Hamza Malik, Zain Zohaib - Alfaaz	80.1	A major (11B)	-13.5	3:04
Alan Walker - Faded	89.9	F# major (2B)	-8.5	3:33
Vasubdhara Das Kay Kay Shaan Loy Mendonsa - It's the time to Disco	132.5	F# minor (11A)	-10.9	5:35
Farhan Akhtar - Socha Hai	134.2	D major (10B)	-10.5	4:06

How good are these numbers? *Faded* is published at 90 BPM in E♭ minor; the app finds 89.9 BPM and F# major, which is E♭ minor's relative major: the same notes, the classic confusion described in Chapter 2. The browser's own MP3 decoder can give slightly different values (a few tenths of a BPM, and loudness a few dB lower because it averages the two channels before measuring), so the guide's tables come from one fixed method. Always check a measured tempo by ear: tap along, and if the app's beat lines drift from the drums, trust the drums.

4 Mixing: by ear, by science, by machine

A mix moves from one track to the next so that the listener never hears a gap or a clash. Three things must be right: the beats line up, the sounds do not fight, and the change comes at a natural moment.

4.1 Beatmatching and its cost

To make track B's beats meet track A's, play B at rate $r = \mathrm{BPM}_A / \mathrm{BPM}_B$. Playing faster also raises pitch, as on a record, by

$$\Delta = 12 \log_2 r \text{ semitones.}$$

A 2 % speed-up is 0.34 semitone, hard to hear; 6 % is a full semitone, and a voice starts to sound wrong. Tempos can also meet at half or double time ($r = 2\,\mathrm{BPM}_A/\mathrm{BPM}_B$ or half of it); the app tries all three and keeps the smallest change. Professional software can change speed without changing pitch (time-stretching); this app deliberately does not, so you hear the physics.

4.2 Keys after the speed change

Because speed moves pitch, it moves the key: B's root shifts by Δ rounded to whole semitones. The app reads B's new Camelot code and scores it against A's (Chapter 2). Two melodies in clashing keys should not overlap: mix on drums, or after the vocals end.

4.3 EQ, crossfade and loudness

- **Bass swap.** Two basslines together are muddy. Bring B in with its low band cut (a shelf below 250 Hz at -26 dB), then swap on a phrase boundary.
- **Equal-power crossfade.** Gains $\cos(x\pi/2)$ and $\sin(x\pi/2)$ keep the summed power constant, so there is no dip in the middle.
- **Loudness matching.** Every track is trimmed to -14 dB RMS before mixing, as a DJ sets the gain knob, and a limiter on the master stops the overlap from clipping.

4.4 Vocals over music: mashups and stems

A mashup plays one song's vocals over another's music. Each deck's *Play* menu offers *vocals only* and *music only*, made by a physical trick: lead vocals are usually mixed to the centre, equal in both speakers, so

$$\text{mid} = \tfrac{L+R}{2} \text{ keeps them,} \qquad \text{side} = \tfrac{L-R}{2} \text{ cancels them.}$$

Bass and kick drum are centred too, so *music only* adds back the mid below 150 Hz, and *vocals only* keeps the mid from 150 Hz to 7 kHz. Reverb, backing vocals and centred instruments leak through, so this is a sketch, good for learning and rehearsing.

For clean parts use *source separation* with Demucs, an open-source neural network (Meta AI, MIT licence) trained on songs together with their separate tracks; it predicts which part of the sound belongs to the voice, drums, bass and the rest:

```
pip install demucs                      # once; about 2 GB with PyTorch
python tools/separate.py "Socha Hai.mp3" "Disco.mp3"
```

It writes `vocals.mp3` and `no_vocals.mp3` for each song in `separated/`; add them to the Library like any track. Then sync, start the vocal on beat one of a phrase of the music, and check on the Camelot wheel that the keys fit.

4.5 How different scenes mix

Scene	Habit, and why
House, techno, EDM	Steady machine tempos (120–128 BPM) and long DJ intros: lock the beats, blend 16–32 bars, swap the bass on a phrase.
English pop, radio	Songs built on vocals; radio edits start singing quickly: short overlaps (about 4 bars) at the end of a chorus.
Bollywood, desi	Often played by live musicians, so tempo drifts; vocals carry the song and big stretches sound wrong: keep stretches small, change on the sam after a fill or a shout, use the dhol as a bridge, cut rather than blend.
Acid house, acid techno	Built on a resonant filter's squelch: close the old track's filter into a throb while the new one opens.
Any	When nothing fits: echo out, then start clean on a downbeat.

These habits are the app's **presets** (`web/mix.js`). Each is a list of automation points (volume, low EQ, filter, echo) placed in bars, so it scales with tempo, plus the largest stretch it accepts: house and acid 8 %, Bollywood 5 %, pop 4 %, echo-out none.

4.6 Automatic choices

With *Auto*, the app picks a preset from the measurements: more than 8 % tempo change even at half or double time gives an echo-out; 4–8 % a clean cut; clashing keys a short crossfade; otherwise a long blend with a bass swap. The order of a set is found greedily: start from the calmest track, then always take the remaining track that scores best on tempo closeness, key fit after the speed change, and a small bonus for being louder, so energy builds. These are rules of thumb written as code: your ear is the judge, and you can override every choice.

5 Building and remaking a set

5.1 Step by step in the app

1. **Library**: press *Add audio files* and choose your tracks, or drop them on the panel. Wait for each card to say *ready*.
2. Read the **pair table** (Figure 5.1): green pairs blend easily, amber with care, red are better cut or echoed out.
3. Tick *in the set* on each track you want.
4. **Set builder**: choose a *Transition style*, or leave *Auto*; press *Auto order*, or move tracks with *Up*.
5. Read each transition (Figure 5.2): where it leaves the old track, the speed and pitch change, the key after it, and why the style was chosen. Open *The rule behind this preset* for the scene's habit.
6. Press *Render MP3* (or WAV). The app plays the whole set through its filters faster than real time, encodes it, and saves the audio and a set file.

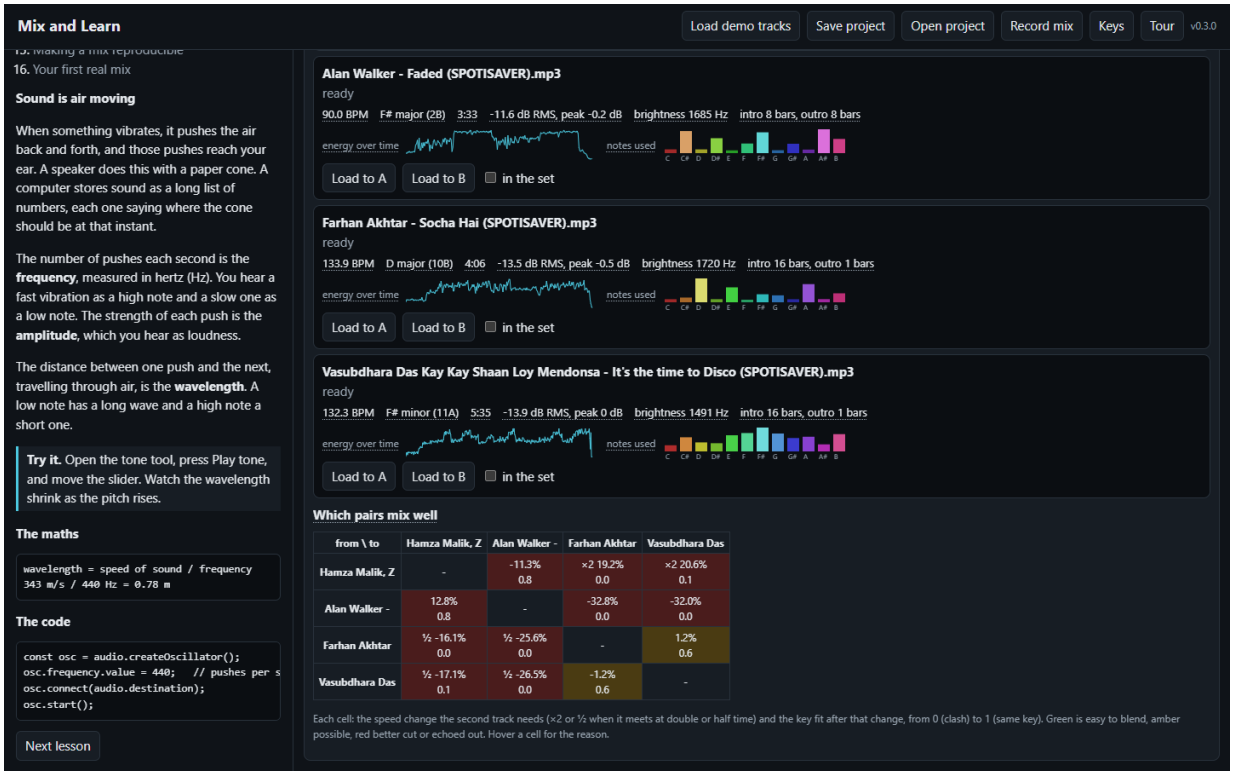


Figure 5.1: How every pair of tracks would mix: speed change for the second track and key fit after it.

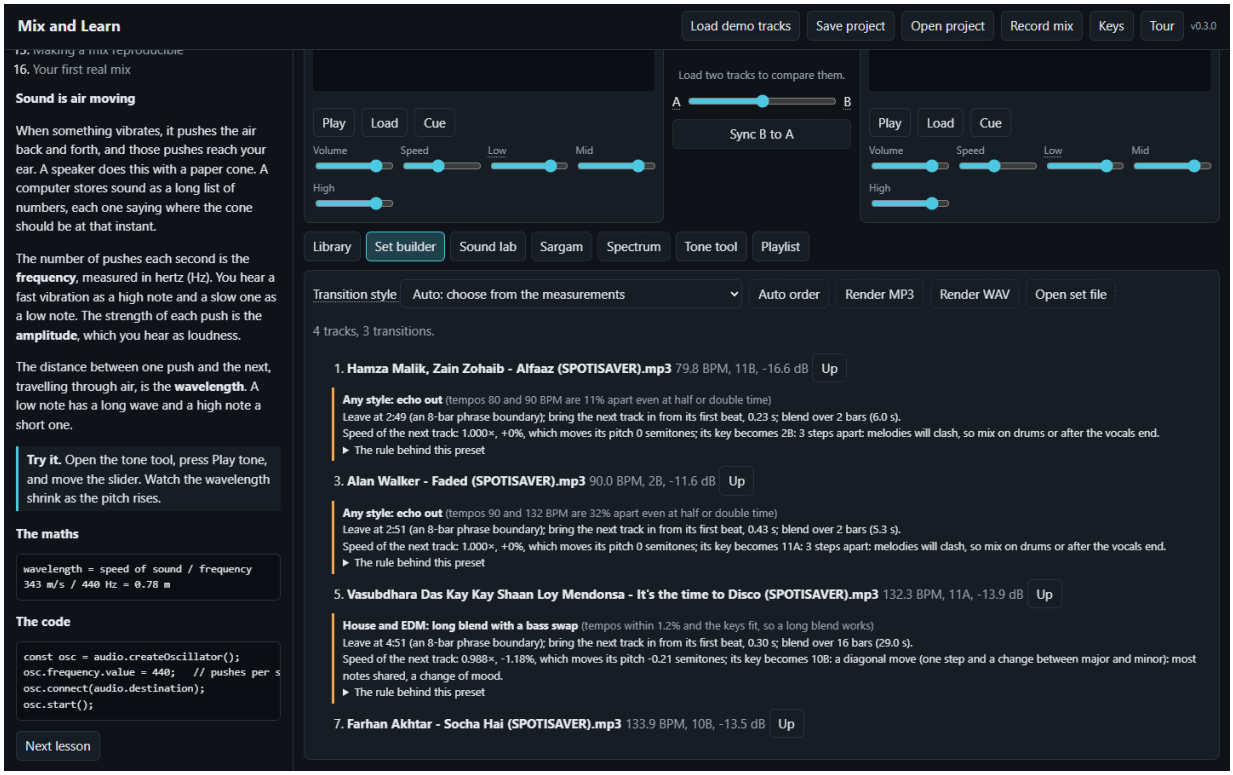


Figure 5.2: The Set builder with the four tracks in automatic order, each transition explained.

5.2 Set 1: Alfaaz and Faded

Track	BPM	Key	RMS (dB)	Length
Hamza Malik, Zain Zohaib - Alfaaz	80.1	A major (11B)	-13.5	3:04
Alan Walker - Faded	89.9	F# major (2B)	-8.5	3:33

1. **Hamza Malik, Zain Zohaib - Alfaaz → Alan Walker - Faded**: Any style: echo out. Leave at 2:48; next track at 1× (0 %), pitch 0 semitones, key after the change 2B: 3 steps apart: melodies will clash, so mix on drums or after the vocals end. *Why:* tempos 80 and 90 BPM are 11% apart even at half or double time.

Tempos 80 and 90 BPM are 11 % apart, and even at half or double time no match is closer, so blending would audibly bend both voices. The plan echoes *Alfaaz* out on a phrase boundary and starts *Faded* clean on its first beat.

5.3 Set 2: Socha Hai and It's the Time to Disco

Track	BPM	Key	RMS (dB)	Length
Farhan Akhtar - Socha Hai	134.2	D major (10B)	-10.5	4:06
Vasubdhara Das Kay Kay Shaan Loy Mendonsa - It's the time to Disco	132.5	F# minor (11A)	-10.9	5:35

1. **Farhan Akhtar - Socha Hai → Vasubdhara Das Kay Kay Shaan Loy Mendonsa - It's the time to Disco**: House and EDM: long blend with a bass swap. Leave at 3:20; next track at 1.013× (1.3 %), pitch 0.22 semitones, key after the change 11A: a diagonal move (one step and a change between major and minor): most notes shared, a change of mood. *Why:* tempos within 1.3% and the keys fit, so a long blend works.

Here the tempos are within about 1.3 %, a change of a fifth of a semitone, and the keys sit diagonally on the wheel, so a long blend works. Both are Bollywood songs: if the blend sounds busy because both have vocals, choose the *Bollywood and desi* style instead, which cuts cleanly on the sam.

5.4 Both sets together

Track	BPM	Key	RMS (dB)	Length
Hamza Malik, Zain Zohaib - Alfaaz	80.1	A major (11B)	-13.5	3:04
Alan Walker - Faded	89.9	F# major (2B)	-8.5	3:33
Vasubdhara Das Kay Kay Shaan Loy Mendonsa - It's the time to Disco	132.5	F# minor (11A)	-10.9	5:35
Farhan Akhtar - Socha Hai	134.2	D major (10B)	-10.5	4:06

1. **Hamza Malik, Zain Zohaib - Alfaaz → Alan Walker - Faded**: Any style: echo out. Leave at 2:48; next track at 1× (0 %), pitch 0 semitones, key after the change 2B: 3 steps apart: melodies will clash, so mix on drums or after the vocals end. *Why:* tempos 80 and 90 BPM are 11% apart even at half or double time.

2. **Alan Walker - Faded → Vasubdhara Das Kay Kay Shaan Loy Mendonsa - It's the time to Disco**: Any style: echo out. Leave at 2:51; next track at 1× (0 %), pitch 0 semitones, key after the change 11A: 3 steps apart: melodies will clash, so mix on drums or after the vocals end. *Why:* tempos 90 and 133 BPM are 32% apart even at half or double time.

3. **Vasubdhara Das Kay Kay Shaan Loy Mendonsa - It's the time to Disco → Farhan Akhtar - Socha Hai**: House and EDM: long blend with a bass swap. Leave at 5:04; next track at 0.9872× (-1.28 %), pitch -0.22 semitones, key after the change 10B: a diagonal move (one step and a change between major and minor): most notes shared, a change of mood. *Why:* tempos within 1.3% and the keys fit, so a long blend works.

The automatic order starts with the calmest track and moves through the slower pair into the faster one. The first two changes are echo-outs because the gap from 90 to 133 BPM cannot be bridged without changing the songs; between the two fast tracks it blends.

5.5 A whole folder at once, and where to start

Choose or drop a `.zip` file of songs on the Library (or on a deck). The app opens it in the browser, takes every audio file inside (MP3, WAV, M4A, FLAC, OGG), skips anything else, measures and remembers each track, and then plans a route through all of them: *Start here* says why the first track was chosen (the calmest, so the set has room to build), and *Why next* gives, for every following track,

its tempo distance, its key fit and whether the energy rises. *Plan from all tracks* does the same at any time.

5.6 Hook to hook

A favourite move in South Asian DJing is to go from one song's hook (the mukhda or chorus) straight into the next song's hook, skipping the verses. The app finds each song's hooks: stretches of 8 bars that repeat later in the song and are loud. Every bar is summarised by its chroma, each 8-bar stretch is compared with every other by cosine similarity, and the score is

$$\text{score} = s^2\,(0.4 + \ell)\,(1 + 0.15\,r),$$

with s the best similarity to another stretch, ℓ the relative loudness and r the number of close repeats (at most 4). The three best stretches that do not overlap are offered as hooks on each Library card and are shaded on the deck waveforms. A repeated verse or a breakdown can outscore the chorus, so the choice is yours: click a hook to hear it, shift-click to use it, or play the song and press *Mark hook* at the start of the real one.

With the style *South Asian hook to hook*, each track plays from the start of its hook to its end, and the next starts on the next downbeat at its own hook, with a quarter-bar crossfade so the cut does not click; a small tempo difference (up to 6 %) is matched. The *hook sequence* shows the whole medley as blocks in time (Figure 5.3). For the four example songs it is 1:15 long.

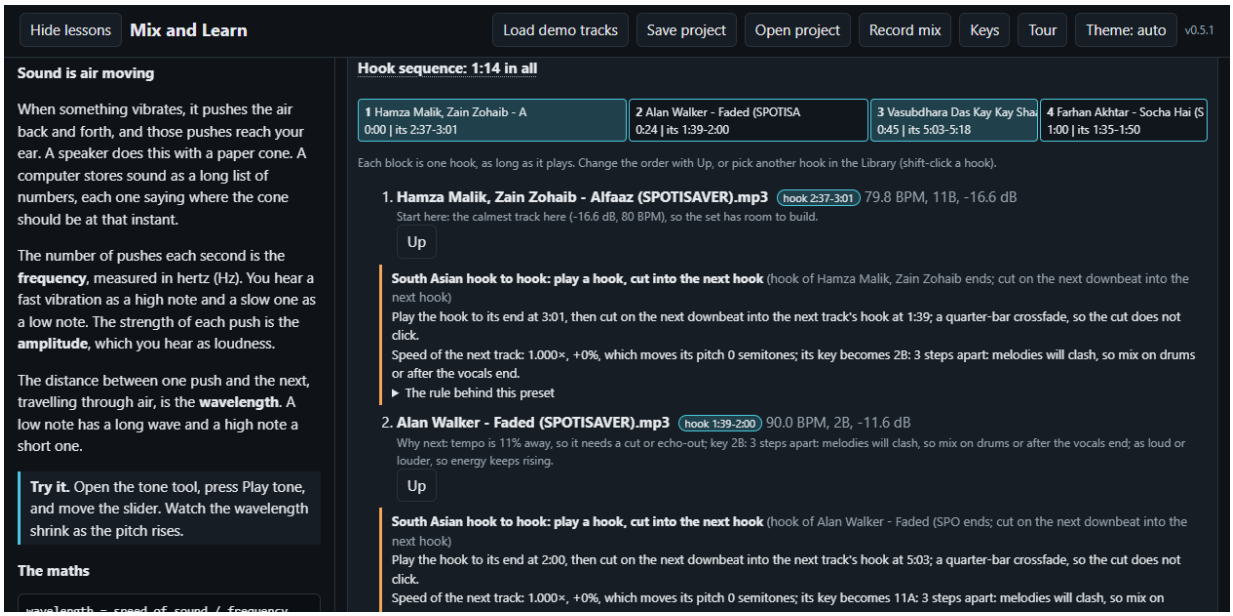


Figure 5.3: A hook medley of the four example songs: the sequence at the top, then each track with why it follows and where it cuts.

5.7 Playlists: one per zip, combined when you want

Every zip you add becomes a playlist named after it; files added on their own go into *Single files*. Everything is remembered on the device, so a playlist added last week is still there. A song that comes in two zips is stored once (it is recognised by its fingerprint) and belongs to both. The chips

above the Library choose what you work with: tick one playlist to see and plan only it, or tick several to combine them, and *Plan from the shown playlists* orders all their tracks as one set by the same musical rules (tempo, key, rising energy). When a new zip arrives, the app shows it and plans a route through it straight away.

5.8 A video for Instagram

Under the set, *Video for Instagram* makes an MP4 of the mix: choose the shape (Story or Reel 9:16, feed portrait 4:5, square 1:1) and the length (60, 90 or 180 seconds). It takes the current set (or a route through the shown playlists), plays each track's hook for 16, 8 or 4 bars, as many tracks as fit the length, renders the audio, then plays it once while drawing every frame and recording picture and sound together. The frame shows the track's title and artist, its tempo, key, speed change and where its hook sits in the song; a live spectrum; the waveform moving past a playhead, coloured by track; what comes next and how the cut is made; and the progress through the mix (Figure 5.4). Recording runs in real time, so a one-minute video takes a minute; keep the tab in front. For the two example playlists combined, a 60-second Story holds three hooks of 8 bars.



Figure 5.4: Three frames of a 60-second Story made from the two example playlists: one per hook, with what comes next and the progress bar.

5.9 The set file and remaking a mix

Rendering also saves `mix_DATE_set.json`. It records the app version, every track's name, length, measurements and **SHA-256 fingerprint** (a 64-character code that changes completely if one byte of the file changes), every transition with its speeds, times and automation points, and the timeline: when each track starts, from where, at what speed and with what loudness trim. To remake the mix, add the same files to the Library, press *Open set file*, and render. The audio itself is never stored in

the set file, so a set can be shared without sharing the music.

From the command line, the same planning runs without a browser:

```
node tools/plan.mjs --order auto file1.mp3 file2.mp3 file3.mp3
node tools/plan.mjs --preset bollywood-cut "Socha Hai.mp3" "Disco.mp3"
```

6 Inside the code

The app is ordinary modern JavaScript in separate modules, loaded by the browser directly. Nothing is compiled, so any file can be opened, read and changed, and the change appears in the browser at once (Chapter 1).

File	What it does
`index.html`, `style.css`	The page and its look; touch-screen and narrow-screen rules at the end of the stylesheet.
`app.js`	Decks, crossfader, sync, live views, lessons, keyboard keys; starts everything.
`dsp.js`	All measurement: notes, YIN pitch, FFT, onsets, tempo, chroma, key, loudness, brightness, sections.
`mix.js`	The DJ's rules as pure functions: tempo matching, key after a speed change, key fit, presets, transition plans, set order, WAV encoding, set files.
`render.js`	Plays a planned set through filters in an OfflineAudioContext; WAV and MP3 output; file fingerprints.
`studio.js`	Library, pair table, Set builder and Sound lab screens.
`help.js`	Explanations for every control and the first-run tour.
`platform.js`	The only difference between desktop and Android: how a file is saved.
`analyse.worker.js`	Runs the analysis in the background so the page stays responsive.
`lessons.js`, `lessons_more.js`	The lessons.
`vendor/lame.min.js`	LAME, the open-source MP3 encoder (LGPL), compiled to JavaScript (lamejs 1.2.1).

6.1 How data flows

A file is read as bytes, fingerprinted (SHA-256), decoded by the browser into samples, reduced to one channel at about 11 kHz, and analysed in the worker. The analysis feeds the Library cards and the pair table. Ticked tracks go to `planTransition` and `orderSet`, whose plans go to `renderSet`; the rendered samples go to the WAV or MP3 encoder, and the plans to the set file.

6.2 Tests

```
npm test        # runs web/test_dsp.mjs and web/test_mix.mjs in Node
```

The tests use *known answers*: synthetic tracks of known tempo and key, pure tones of known pitch, set rules with hand-checked results (70 and 140 BPM meet at double time; A minor up a semitone is 3A). A change that breaks a rule fails a test before it reaches a user.

6.3 Adding a preset

A preset is an entry in `PRESETS` in `mix.js`: a name, a length in bars, the largest stretch it accepts, the rule in words, and a function returning automation points. Each point has a time in bars and any of: `aGain`, `bGain` (volume 0 to 1), `aLow`, `bLow` (low EQ in dB), `aCut` (A's low-pass frequency), `bHigh` (B's high-pass frequency), `aEcho` (echo feedback). Add it, reload, and it appears in the Set builder.

6.4 Open-source parts and their science

- **Web Audio API** (in every browser): audio graphs of sources, filters (biquad: two poles and two zeros, the standard EQ building block), gains, delays and compressors, played live or rendered offline.
- **LAME** via lamejs: MP3 compression uses a psychoacoustic model to discard what the ear cannot hear next to louder sounds (masking).
- **Krumhansl–Kessler key profiles** (1982): listening experiments on how well each note fits a key.
- **YIN** (de Cheveigné and Kawahara, 2002): pitch from the self-similarity of a wave.
- **Capacitor** (Ionic): wraps a web app in a native Android or iOS shell (Chapter 7).

7 The Android app, from nothing

One guide is enough for both versions, because there is one app: Android shows the same `web/` folder inside a native shell. This chapter explains the ideas, then every step from a fresh computer to the app on your phone.

7.1 The ideas

APK An Android Package: a zip file holding an app's code, resources and a manifest describing it. Phones install APKs.

Android SDK The Software Development Kit: tools that build APKs and talk to phones. It comes with Android Studio.

Gradle The build tool Android projects use: it compiles, packs and signs the APK. `gradlew` is a small script that downloads the right Gradle version by itself.

WebView A browser engine inside an app. Our app's screens are web pages shown in a WebView.

Capacitor An open-source bridge (by Ionic) that creates the Android project around a web folder, copies the web files into it, and gives JavaScript access to phone features through *plugins*, here Filesystem (to save the rendered mix) and Share (to send it to another app).

Signing Every APK is signed with a key. Debug builds use an automatic debug key; apps for others need your own release key, kept safe, because updates must be signed with the same key.

7.2 What is already done in this repository

```
npm install                # Capacitor and its plugins, listed in package.json
npx cap add android        # created the android/ folder (done once, committed)
npx cap sync android       # copies web/ into the Android project; run after every change
```

The file `capacitor.config.json` names the app (*Mix and Learn*), the folder of web files (`web`), and the app's unique id, a reversed web address:

```
io.github.ahmadammar.mixandlearn
```

7.3 Your steps: from a fresh computer to the phone

1. **Install Android Studio** from developer.android.com/studio. Its first-run wizard installs the Android SDK, platform tools and a Java runtime for Gradle (Android's build tools need Java 17 or 21; Android Studio bundles one, so a newer Java elsewhere on your computer does not matter).

2. **Install Node.js** (nodejs.org, the LTS version), if it is not installed.

3. In the `djmixer` folder run `npm install`, then `npx cap sync android`.
4. **Open the project**: `npx cap open android` opens `android/` in Android Studio. Wait for *Gradle sync* to finish (the first time it downloads a few hundred megabytes).
5. **Prepare the phone** (Samsung and most Android phones): Settings → About phone → Software information → tap *Build number* seven times to enable Developer options; then Settings → Developer options → turn on *USB debugging*. Connect the phone by USB and accept the prompt on its screen.
6. **Run**: in Android Studio choose your phone in the device list and press the green Run button. The app installs and opens.
7. **Or build an APK file**: Build → Build App Bundle(s) / APK(s) → Build APK(s), or in a terminal:

```
cd android
gradlew assembleDebug
# the file: android/app/build/outputs/apk/debug/app-debug.apk
```

   Copy the APK to the phone and open it; allow installation from this source when asked.
8. **A release APK** for others: Build → Generate Signed Bundle / APK, create a keystore (keep it and its password safe and out of git), and choose *release*.

7.4 Keeping desktop and phone in step

Change files only in `web/`; test on the desktop with `python dev/serve.py`; then `npx cap sync android` and run again on the phone. The `android/` folder holds only the shell. The one place where code differs by platform is `web/platform.js`: in a browser a rendered mix is downloaded; on the phone it is written to Documents/MixAndLearn and offered to the share sheet.

7.5 Designing for touch

- **Targets**: on touch screens every button and slider is at least 44 pixels tall, the size of a fingertip (the `pointer: coarse` rules in `style.css`).
- **No hover**: explanations open on press-and-hold (half a second) instead.
- **Narrow screens**: below 700 pixels the layout becomes one column, the tabs scroll sideways, and the EQ knobs sit three to a row (Figure 7.1).
- **Battery and memory**: analysis runs in a background worker at about 11 kHz; a phone handles a few tracks comfortably, but rendering a long set takes longer than on a computer.

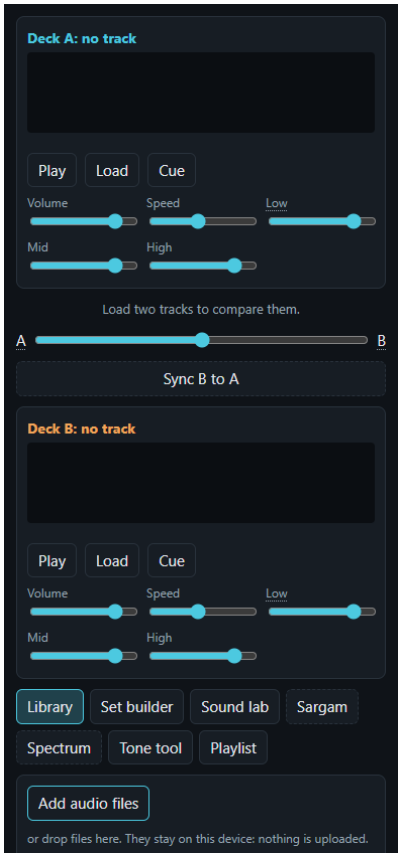


Figure 7.1: The same app at a phone's width (390 pixels), in the desktop browser's phone view.

7.6 Do I need a second guide?

No. Android development often means learning Kotlin or Java and Android's own layout system; here the app is a web app, so the only Android-specific knowledge is in this chapter: install the tools, sync, run, sign. If a future version needs features a WebView cannot give (for example, low-latency audio for live scratching), that would be the point to add native code, and a separate chapter.